

```
In [1]: # Rode o comando abaixo para poder usar os códigos em cpp

!mkdir cpp_files
```

Arrays Unidimensionais

Até agora, trabalhamos com variáveis que armazenam apenas um valor por vez. Isso funciona bem em muitos casos, mas logo aparece uma limitação: em vários problemas precisamos guardar múltiplos valores do mesmo tipo. Por exemplo, 5 leituras de um sensor, 10 notas de alunos ou várias distâncias medidas ao longo de um percurso. Fazer isso com variáveis separadas rapidamente deixa o código repetitivo e difícil de organizar. Para resolver esse problema, utilizamos arrays.

O que é um array?

Um array é uma estrutura que permite armazenar vários valores do mesmo tipo em sequência. Cada elemento ocupa uma posição, e cada posição pode ser acessada por um índice.

Podemos imaginar um array assim:

```
[ 12 ][ 8 ][ 25 ][ 30 ][ 17 ]
   0   1   2   3   4
```

A ideia principal é simples: em vez de criar muitas variáveis separadas, agrupamos tudo em uma única estrutura e acessamos cada valor pela sua posição.

Declaração e inicialização

A sintaxe básica de um array em C++ é:

```
tipo nome[tamanho];
```

Por exemplo:

```
int valores[5];
```

Esse array foi criado para armazenar 5 números inteiros. Também podemos inicializá-lo diretamente:

```
int valores[5] = {10, 20, 30, 40, 50};
```

Acesso e modificação

Para acessar um elemento, usamos o nome do array seguido do índice entre colchetes. Para modificar, fazemos o mesmo. Vamos analisar um exemplo com inicialização, acesso e

modificação:

```
In [ ]: %%writefile cpp_files/primeiro_exemplo.cpp
#include <iostream>
using namespace std;

int main() {

    int valores[5] = {10, 20, 30, 40, 50};

    cout <<"Primeiro valor do array: "<<valores[0] << endl;
    cout<<"Terceito elemento do array antes de alterar: "<<valores[2]<<endl;
    valores[2] = 99; // modifica o terceiro elemento
    cout <<"Terceito elemento do array após alterar: "<< valores[2] << endl;

    return 0;
}
```

Overwriting cpp_files/primeiro_exemplo.cpp

O Tamanho do Array (sizeof)

Diferente de outras linguagens, o array comum em C++ não possui uma função `.length()`. Para descobrir quantos elementos um array tem, usamos o operador `sizeof`, que retorna o tamanho em bytes.

O método mais usual é dividir o tamanho total do array pelo tamanho de apenas um elemento:

```
In [ ]: %%writefile cpp_files/segundo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    double dados_gps[6] = {-23.55, -46.63, 10.5, 12.8, 5.5, 0.1};

    // Calculando o numero de elementos
    int total_bytes = sizeof(dados_gps);
    int tamanho_um_item = sizeof(dados_gps[0]);
    int quantidade = total_bytes / tamanho_um_item;

    cout << "Tamanho total em bytes: " << total_bytes << endl;
    cout << "Cada double ocupa: " << tamanho_um_item << " bytes" << endl;
    cout << "Quantidade de elementos no array: " << quantidade << endl;

    return 0;
}
```

Omitindo o Tamanho na Inicialização

Muitas vezes, já sabemos quais valores queremos colocar no array logo no início. Nesses casos, o C++ nos permite omitir o número dentro dos colchetes `[]`. O compilador contará os elementos dentro das chaves `{}` e definirá o tamanho automaticamente.

```
In [ ]: %%writefile cpp_files/terceiro_exemplo.cpp
#include <iostream>
#include <string> //Vocês ainda não viram strings, mas confio que entenderão
using namespace std;

int main() {
    // O C++ entende que este array tem tamanho 4 automaticamente
    int array[] = {1, 2, 3, 4};

    // Calculando o numero de elementos
    int total_bytes = sizeof(array);
    int tamanho_um_item = sizeof(array[0]);
    int quantidade = total_bytes / tamanho_um_item;

    cout<<"A quantidade de elementos no array eh: "<<quantidade<<endl;

    cout << "O primeiro número da lista e: " << array[0] << endl;
    cout << "O ultimo número da lista e: " << array[3] << endl;

    return 0;
}
```

Writing cpp_files/terceiro_exemplo.cpp

Riscos – Acesso Fora dos Limites (Out of Bounds)

Este é um dos erros mais comuns e perigosos. Se você tem um array de tamanho 5, os índices válidos são de 0 a 4. Tentar acessar o índice 5 não causará um erro de compilação, mas fará o programa ler um local da memória que não pertence ao array.

Atenção: Isso pode retornar valores "lixo" ou até travar o sistema do seu robô (Segmentation Fault).

```
In [ ]: %%writefile cpp_files/quarto_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    int meu_array[3] = {10, 20, 30};

    // Acessando um indice que nao existe!
    cout << "Indice 0: " << meu_array[0] << endl;
    cout << "Indice 10 (PERIGO): " << meu_array[10] << endl;

    return 0;
}
```

Percorrendo Arrays com Loops

Como vimos na Aula 4, os loops são ferramentas poderosas para repetir tarefas. Com arrays, eles são indispensáveis para processar todos os dados de uma vez.

O for tradicional

É a melhor escolha quando você precisa saber o índice (a posição) de cada elemento. Note que é preciso prestar atenção para não acessar um endereço de memória inexistente, gerando uma leitura errônea, como descrito no bloco anterior.

O Range-based for (C++11)

É uma forma mais moderna e limpa de percorrer o array do início ao fim quando você só precisa dos valores.

```
In [ ]: %%writefile cpp_files/quinto_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    int leituras[] = {15, 22, 18, 25, 30};

    // 1. Usando o for tradicional (precisamos saber o tamanho: 5)
    cout << "Percorrendo com for tradicional:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "Indice " << i << " contem valor: " << leituras[i] << endl;
    }

    cout << "-----" << endl;

    // 2. Usando range-based for (mais simples e elegante)
    cout << "Percorrendo com range-based for:" << endl;
    for (int valor : leituras) {
        cout << "Valor lido: " << valor << endl;
    }
}
```

```
    return 0;
}
```

Preenchimento de Arrays com Loops e Inputs

Até agora, escrevemos os valores dos arrays diretamente no código. No entanto, na robótica, os dados virão frequentemente de sensores ou de configurações que o utilizador define na hora. Para automatizar esse processo, utilizamos o loop `for`.

Existem duas formas principais de gerir o tamanho do array ao preenchê-lo:

1. Tamanho Pré-definido (Constante)

Esta é a abordagem mais segura quando o hardware tem limites fixos (por exemplo, um robô que tem sempre 4 motores). Usamos uma constante (`const`) para que o código seja fácil de manter.

```
In [ ]: %%writefile cpp_files/sexta_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    const int QTD_MOTORES = 4; // Tamanho fixo e protegido
    float velocidades[QTD_MOTORES];

    cout << "Configuracao de velocidade para " << QTD_MOTORES << " motores."

    // Preenchendo o array via loop
    for (int i = 0; i < QTD_MOTORES; i++) {
        cout << "Defina a velocidade do motor " << i << ": ";
        cin >> velocidades[i];
    }

    cout << "\nVelocidades armazenadas com sucesso!" << endl;
    return 0;
}
```

2. Tamanho Definido pelo Utilizador (Input)

Em cenários mais flexíveis, o programa só saberá quantos dados processar durante a execução. Primeiro capturamos o tamanho desejado e depois declaramos o array com essa variável.

```
In [ ]: %%writefile cpp_files/setimo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    int num_leituras;
```

```

cout << "Quantas leituras de sensor deseja realizar nesta rodada? ";
cin >> num_leituras;

// O array é criado com o tamanho que o utilizador digitou
float dados_sensor[num_leituras];

for (int i = 0; i < num_leituras; i++) {
    cout << "Introduza o valor da leitura " << i << ": ";
    cin >> dados_sensor[i];
}

cout << "\nProcessamento de " << num_leituras << " dados concluido." <<
return 0;
}

```

Arrays Multidimensionais

Se um array unidimensional é uma linha de dados, um array multidimensional (ou matriz) é como uma tabela com linhas e colunas. Na robótica, usamos matrizes para representar tudo o que tem uma estrutura de grade, como mapas, imagens de câmeras ou tabelas de coordenadas.

A sintaxe utiliza dois pares de colchetes: `tipo nome[linhas][colunas];`.

```

In [ ]: %%writefile cpp_files/oitavo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    // Declarando uma matriz 2x3 (2 linhas e 3 colunas)
    // Exemplo: 2 robôs, cada um com coordenadas (x, y, z)
    float posicoes[2][3] = {
        {1.2, 0.5, 0.0}, // Linha 0 (Robô A)
        {2.5, -1.0, 0.2} // Linha 1 (Robô B)
    };

    // Acessando um valor específico: Linha 1, Coluna 0 (X do Robô B)
    cout << "Coordenada X do Robo B: " << posicoes[1][0] << " metros." << endl;

    return 0;
}

```

Percorrendo Matrizes com Loops Aninhados

Para processar todos os dados de uma matriz, precisamos de um loop dentro de outro loop (nested loops), como vimos na Aula 4.

- O loop externo controla as linhas.

- O loop interno percorre as colunas daquela linha específica.

```
In [ ]: %%writefile cpp_files/nono_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    int mapa[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    cout << "Imprimindo a matriz completa:" << endl;

    for (int i = 0; i < 3; i++) {           // Loop das linhas
        for (int j = 0; j < 3; j++) {       // Loop das colunas
            cout << "[" << mapa[i][j] << " ";
        }
        cout << endl; // Quebra de linha ao fim de cada linha da matriz
    }

    return 0;
}
```

Arrays de 3 Dimensões (Exemplo de Câmera RGB)

Embora menos comuns para iniciantes, o C++ permite criar arrays com quantas dimensões você precisar. Um exemplo clássico na robótica é o processamento de imagens. Uma imagem colorida é uma matriz de pixels, mas cada pixel possui 3 camadas de cor: Red (Vermelho), Green (Verde) e Blue (Azul).

```
In [ ]: %%writefile cpp_files/decimo_exemplo.cpp
#include <iostream>
using namespace std;

int main() {
    // Uma imagem minúscula de 2x2 pixels, com 3 canais de cor (RGB)
    // Estrutura: imagem[altura][largura][canal]
    int imagem[2][2][3] = {
        { {255, 0, 0}, {0, 255, 0} }, // Linha 0: Pixel(0,0) é Vermelho, Pixel(0,1) é Verde
        { {0, 0, 255}, {255, 255, 255} } // Linha 1: Pixel(1,0) é Azul, Pixel(1,1) é Branco
    };

    cout << "Intensidade de Vermelho no pixel (0,0): " << imagem[0][0][0] << endl;
    cout << "Intensidade de Azul no pixel (1,0): " << imagem[1][0][2] << endl;

    return 0;
}
```